Kubernetes Autoscaling

pods follow the metric, nodes follow the pods

Watch the 4-min video

In one line: pod autoscalers (HPA, VPA, KEDA) decide how many pods you need and how big they are; node autoscalers (Cluster Autoscaler, Karpenter) add machines when pods are **Pending** and remove them when they sit idle. Scale on the signal that really measures the backlog, and remember that nodes, especially GPU nodes, take minutes to arrive.

1 Peaks and valleys

An AI inference service gets **50 requests a minute** overnight and **5,000** at 9 a.m. Sizing GPU pods for the peak all day wastes most of the money; sizing for the night means a queue that takes an hour to clear. Kubernetes solves this at **two levels**: the pod level (how many replicas, how big) and the node level (how many machines to run them on). The two are separate controllers that only meet through one thing: pods that can't be scheduled.

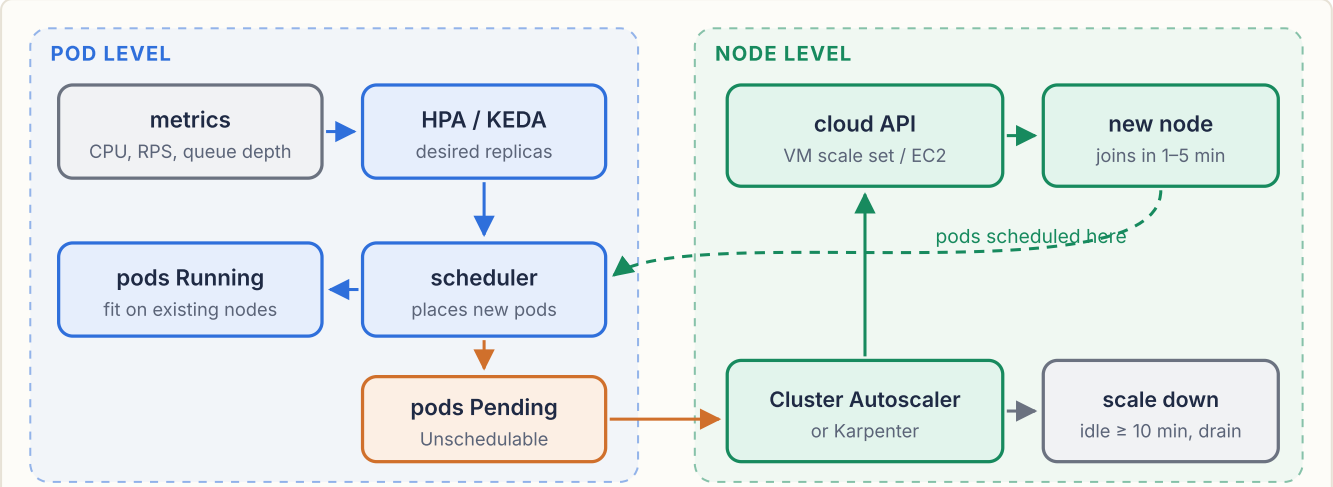


Figure 1. The two-level loop. Pod autoscalers create pods; pods that don't fit stay **Pending**, which is the only signal the node autoscaler reacts to.

2 Horizontal Pod Autoscaler: how many pods

The HPA is a control loop (every **15 s** by default) that reads a metric and sets `replicas` on a Deployment or StatefulSet. Its core formula is:

```
desired = ceil( current replicas × current metric ÷ target metric )
```

With 4 pods at 90% CPU and a 60% target: ceil(4 × 90 ÷ 60) = **6 pods**. Important details:

- **CPU utilisation is a percentage of the pod's request**, not of the node. Pods without CPU requests can't be scaled on CPU at all (lesson 57).
- A **tolerance** of 10% stops it reacting to tiny changes; `minReplicas` and `maxReplicas` bound the result.
- Scale-up is fast; scale-down waits for a **stabilisation window** (300 s by default) and uses the highest recommendation in that window, so a brief dip doesn't remove pods you need a minute later. The `behavior` field tunes both directions (for example, at most 50% fewer pods per minute).

- The `autoscaling/v2` API accepts **resource** metrics (CPU, memory, from metrics-server), **custom** per-pod metrics (requests per second via Prometheus Adapter) and **external** metrics (a queue length in a cloud service). With several metrics, it takes the largest result.

CPU is a good signal for CPU-bound request handlers. It is a poor one for queue workers (an idle worker and a blocked worker both show low CPU) and for GPU work, where the CPU may be nearly idle while the GPU is saturated.

3 Vertical Pod Autoscaler: how big is a pod

The VPA watches actual usage over days and recommends CPU and memory **requests**. It has three parts: a **recommender** (computes targets from usage history), an **updater** (evicts pods whose requests are far off) and an **admission webhook** (rewrites requests when pods are recreated). `updateMode: Off` only publishes recommendations, which is the safe way to start; `Initial` applies them at pod creation; `Recreate` / `Auto` evict running pods to apply them. Newer Kubernetes versions can resize pods in place, which removes the restart for many changes.

Use VPA for right-sizing, and for things that can't scale out (a singleton, a stateful component). **Don't** let HPA and VPA both act on CPU or memory for the same workload: VPA raises requests, so utilisation % drops, so HPA removes pods, so usage per pod rises, and they chase each other. Combining is fine when HPA scales on a different metric (RPS, queue depth).



Figure 2. Four autoscalers, four questions. Most production clusters run HPA or KEDA plus a node autoscaler; VPA mostly in recommendation mode.

4 Pods need nodes: Cluster Autoscaler and Karpenter

The **Cluster Autoscaler** (CA) checks about every 10 s for pods marked *Unschedulable*. It simulates the scheduler against each node pool's template and grows the pool whose nodes would fit them (an *expander* such as least-waste, priority or price picks between pools). For scale-down it looks for nodes whose pods' **requests** add up to under 50% of capacity for 10 minutes (both defaults), checks that those pods can move elsewhere, then drains the node respecting PodDisruptionBudgets. Pods with local storage, no controller, or the `safe-to-evict: "false"` annotation block removal.

Two consequences surprise people. First, the CA works on **requests, not real usage**: a cluster at 15% real CPU but 95% requested is "full". Second, it can only add nodes of the shapes you pre-defined in node pools.

Karpenter (created by AWS, now also behind AKS node auto-provisioning) skips node groups. It reads the pending pods' requests, selectors and tolerations, chooses the cheapest instance types that fit them from a broad allowed list (including spot), and launches those machines directly. It also **consolidates**: it replaces several half-empty nodes with fewer or cheaper ones.

	Cluster Autoscaler	Karpenter
Unit it scales	pre-defined node pools / groups	individual instances chosen per batch of pods
Instance choice	fixed per pool	any type allowed by a NodePool's requirements
Scale-down	utilisation < threshold for N minutes	empty-node removal + consolidation onto cheaper nodes
Where	every major cloud	AWS; Azure (node auto-provisioning); others via providers

5 KEDA: scale on the backlog, and to zero

For queue consumers the right question is "how much work is waiting?". **KEDA** (Kubernetes Event-Driven Autoscaling) has scalers for dozens of sources: Kafka consumer lag, RabbitMQ, SQS, Azure Service Bus, Prometheus queries, cron schedules. A `ScaledObject` names the target and a per-replica target such as **20 messages per worker**. KEDA itself handles **0 ↔ 1** (activation and scale to zero after a cooldown) and creates an HPA behind the scenes for **1 ↔ N**, fed through the external metrics API. For long jobs that must not be killed mid-way, a `ScaledJob` starts one Kubernetes Job per message instead of scaling a Deployment.

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata: { name: inference-workers }
spec:
  scaleTargetRef: { name: inference-worker }   # the Deployment
  minReplicaCount: 2          # warm minimum (0 = scale to zero)
  maxReplicaCount: 120        # cost ceiling
  pollingInterval: 15         # seconds between queue checks
  cooldownPeriod: 600         # only used when dropping to zero
  triggers:
  - type: azure-servicebus
    metadata: { queueName: inference, messageCount: "20" }
```

Figure 3. A KEDA ScaledObject for the worked example. The `messageCount` target becomes the HPA's per-pod average.

6 GPU nodes need extra care

A new GPU worker often needs a new node, and every step of that chain is slow. Plan for the whole chain, not just the HPA reaction:

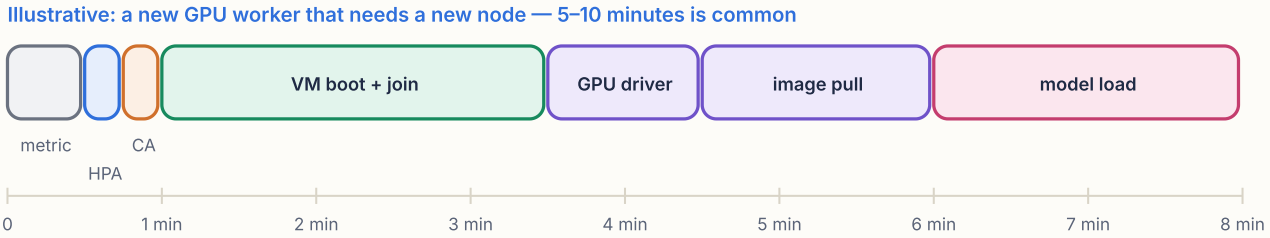


Figure 4. Why GPU scale-up takes minutes. Pod autoscaling reacts in under a minute; the node, drivers, multi-GB image and model weights dominate.

- **Warm minimum:** keep `minReplicaCount` > 0 for latency-sensitive paths. A cheaper trick is **overprovisioning**: low-priority placeholder pods reserve a spare node and are evicted the moment a real pod needs the room, so the next real pod starts in seconds.

- **Separate GPU pools with taints** (e.g. `nvidia.com/gpu:NoSchedule`) so ordinary pods never land on a $3/hour node and keep it from scaling down. GPU pods request `nvidia.com/gpu: 1` and tolerate the taint.
- **Scale on queue depth or in-flight requests**, not CPU. GPU utilisation can be used via DCGM exporter metrics, but backlog maps more directly to user wait.
- **Slow scale-down:** longer cooldowns and node-removal delays avoid paying a 6-minute start for a node you deleted 5 minutes earlier.
- **Faster starts:** pre-pull images into the node image, cache model weights on local NVMe or a shared volume, use smaller images.

WORKED EXAMPLE: INFERENCE WORKERS ON QUEUE DEPTH

Requests go to a queue. KEDA targets **20 messages per worker**, min 2, max 120. Overnight the queue holds about 30 messages, so ceil(30 ÷ 20) = 2 workers, the floor. At 9 a.m. the queue reaches **2,000**: ceil(2,000 ÷ 20) = **100** workers. The HPA's default scale-up policy lets replicas at most double (or +4) every 15 s, so 2 → 4 → 8 → … reaches 100 within a couple of minutes. Most new pods are Pending: if each GPU node runs 4 workers (one GPU each), about 24 more nodes are needed, and the CA or Karpenter requests them. The first new nodes are serving after 5–10 minutes; the 2 warm workers and the overprovisioned spare node absorb the start. In the evening the queue drains; after the scale-down stabilisation window the workers return to 2, nodes become empty, and after the scale-down delay the node autoscaler deletes them.

COMMON MISTAKES

- No resource requests: HPA can't compute CPU %, and the CA can't tell what fits.
- HPA and VPA both on CPU for the same Deployment, fighting each other.
- Scaling queue workers on CPU, so a growing backlog of slow I/O work never triggers scale-out.
- No `maxReplicas` or node pool maximum: a bug or traffic spike becomes an unbounded bill.

TRADE-OFFS TO STATE

- **Scale to zero** saves the most money but adds a cold start of minutes for GPU work.
- **Aggressive scale-down** saves cost; **slow scale-down** saves latency and churn.
- **Headroom** (warm pods, placeholder pods) costs a little all day to avoid a slow first minute at every spike.
- Predictable peaks (9 a.m. daily) can also be met with a **cron** trigger that scales up at 8:45.

7 Key terms

HPA: changes replica count to keep a metric near its target

Cluster Autoscaler: adds nodes for Pending pods; removes underused nodes

KEDA: event-driven scaling on queues, lag, cron; supports zero

VPA: changes pods' CPU/memory requests from observed usage

Karpenter: provisions right-sized instances directly for pending pods

Stabilisation window: delay that smooths scale-down decisions

8 Interview questions

Q1. How do pod and node autoscaling interact? BASIC

HPA or KEDA change the replica count; pods that can't fit stay Pending, which triggers the Cluster Autoscaler or Karpenter to add nodes; nodes that stay underused are drained and removed later.

Q2. What should queue workers scale on? BASIC

Queue depth or consumer lag, using KEDA (or an external-metric HPA), not CPU.

Q3. How does the HPA compute the replica count? BASIC

desired = ceil(current replicas × current metric ÷ target), within min/max, ignoring changes within a 10% tolerance, with a stabilisation window on scale-down.

Q4. Your cluster shows 20% real CPU usage but the Cluster Autoscaler keeps adding nodes. Why? SCENARIO

It schedules on requests, not usage: pods request far more than they use. Right-size requests (VPA recommendations) and the nodes will pack tighter.

Q5. Users wait 8 minutes when traffic jumps at 9 a.m. on your GPU service. What do you change? SCENARIO

Keep a warm minimum or placeholder pods for one spare node, add a cron pre-scale before 9, pre-pull images and cache model weights to cut start time, and scale on queue depth so scale-out starts early.

Q6. Why not run HPA and VPA together on CPU? TRICKY

VPA changes requests, which changes the utilisation percentage HPA sees, so the two controllers keep reversing each other; use VPA for sizing and HPA on a different metric.

Q7. Why doesn't the Cluster Autoscaler remove an obviously idle node? TRICKY

Something on it blocks eviction: a PodDisruptionBudget with no room, a pod with local storage or no controller, a safe-to-evict=false annotation, or a system pod that can't move.

Q8. How does KEDA scale to zero if the HPA can't go below one? TRICKY

KEDA handles activation itself: it watches the source, scales the Deployment between 0 and 1 directly, and lets its managed HPA handle 1 to N.

Next: SD-V-009 · Stateful Workloads on Kubernetes backendroadmap.com